

第16章 类型转换

16.1 类型转换

类型转换主要包括三种情形：内置类型之间的转换、内置类型与自定义类型之间的转换，以及自定义类型之间的转换。

当需要进行内置类型与内置类型间的转换时，可以通过类型提升和强制类型转换实现。如：

```
1  int main( ) {
2      int n = 100;
3      double val = n;           //类型提升
4      double * p = (double *)(&n); //强制类型转换
5  }
```

当需要将内置类型转换为自定义类型时，可以通过自定义类型的转换构造函数来实现；而将自定义类型转换为内置类型，则可以通过自定义类型中的自动转换函数来完成。在类中定义转换构造函数或自动转换函数时，如果不使用 `explicit` 关键字，默认情况下会启用隐式转换功能，即当类型不匹配时，编译器会自动尝试使用这些转换功能。如果希望禁用隐式转换，可以使用 `explicit` 关键字，明确禁止编译器自动进行类型转换。例如：

```
1  class Fraction {
2  public:
3      explicit Fraction( long num = 0, unsigned long den = 1 ):mNum( num ), mDen( den ){ }
4      operator long( ) const { return mNum / mDen; }
5      explicit operator double( ) const { return ( ( double ) mNum ) / mDen; }
6  public:
7      void testCtr( const Fraction & frac ) {
8          std::cout << frac.mNum << "/" << frac.mDen << std::endl;
9      }
10     void testLong( long n ) { std::cout << n << std::endl; }
11     void testDouble( double val ) { std::cout << val << std::endl; }
12 private:
13     long          mNum;    //分子
14     unsigned long mDen;    //分母
15 };
16 int main( ) {
17     //通过转换构造函数，将数值 3 转换成 Fraction 对象
18     Fraction v1( 3 );
19     //构造分数 8/5
20     Fraction v2( 8, 5 );
```

```

21    //通过自动转换函数将分数转内置类型
22    long n1 = v1.operator long( );    //n1=3
23    long n2 = ( long ) v1;          //n2=3
24    long n3 = long( v1 );           //n3=3
25    //通过自动转换函数将分数转内置类型
26    double m1 = v2.operator double( ); //m1=1.6
27    double m2 = ( double ) v2;       //m2=1.6
28    double m3 = double( v2 );        //m3=1.6
29    //v1.testCtr( 2 );                //错误，无法将 2 隐式转换成分数
30    v1.testCtr( Fraction( v2 ) );    //输出 8/5
31    v1.testCtr( Fraction( 4,3 ) );   //输出 4/3
32    v1.testLong( v2 );               //输出 1
33    v1.testDouble( v2 );             //输出 1，过程中 v2 先隐式转成 long，再类型提升为 double
34 }

```

对于自定义类型之间的转换，可以通过定义转换构造函数或自动类型转换函数来实现。若要实现从类 A 到类 B 的转换，可以在类 A 中定义一个自动转换函数 `operator B( )`，或者在类 B 中定义一个接受类 A 对象的构造函数 `B(A& a)`。然而，这两种方法不能同时存在，否则会导致编译时的二义性错误。例如：

```

1  class A;
2  class B {
3  public:
4      //explicit B( const A & ) { } //与 A::operator B( ), 只能二选一
5      B( int n ) { }
6      /* 其它略 */
7  };
8  class A {
9  public:
10     explicit operator B( ) const { return B( 2 ); }
11     /* 其它略 */
12 };
13 void testA2B( const B & aB ) { }
14 int main( ) {
15     A aA;
16     //需要 aA 转 B 类对象，
17     testA2B( (B)aA ); //显式转换，合法
18     testA2B( B(aA) ); //显式转换，合法
19     //testA2B( aA ); //隐式转换，非法
20 }

```

存在继承关系时，基类、派生类及后裔类共同构成一棵继承树，树中的各类之间可以进行类型

转换。这里可以分为两种情况来讨论：一种是在 `private` 或 `protected` 继承方式下的类型转换，另一种是 `public` 继承方式下的类型转换。

16.2 私有或保护继承下的类型转换

在私有或保护继承的情况下，基类和派生类中的公有成员函数会表现出完全不同的接口特征。此时如果尝试进行类型转换，实际上就像是在转换两个毫无关联的实体，这在逻辑上是不合理且没有意义的。因此，在私有或保护继承中，若要实现基类与派生类之间的相互转换，必须显式地定义转换函数，比如自定义转换构造函数或自动类型转换函数。例如：

```

1  class Base {
2  public:
3      Base( int n ) : mNum1( n ) { }
4  private:
5      int mNum1;
6  };
7  class Derived : private Base {
8  public:
9      Derived( int n1, int n2 = 0 ) :Base( n1 ), mNum2( n2 ) { }
10     Derived( const Base & base ) :Base( base ), mNum2( 0 ) { }
11     operator Base( ) const { return *( Base * ) ( this ); }
12 private:
13     int mNum2;
14 };
15 int main( ) {
16     Base base( 1 );
17     Derived d1( base );      //合法 Base 转 Derived
18     Derived d2 = base;      //合法 Base 转 Derived
19     Base b1 = d1.operator Base( ); //合法 Derived 转 Base
20     //Base b2 = ( Base ) d1; //部分编译器非法
21     //Base b3 = Base (d1);   //部分编译器非法
22 }
```

16.3 公有继承下的类型转换

在公有继承中，父类和子类的公有成员存在交集。具体来说，子类的公有成员完全包含了父类的公有成员，这表明子类扩展了父类的行为集合，而父类则泛化或抽象了子类的行为集合。这种关系体现了子类是对父类类型的进一步细化，是父类的子类型化结果。因此，在这种分类的逻辑下，父类和子类之间的相互转换具有实际意义。

16.3.1 向上类型转换

将子类转换为父类称为向上类型转换。在公有继承中，父类是子类的抽象泛化。从逻辑上讲，将子类视为父类总是合理的，如，可以将苹果或橘子视为水果，将大学生或中学生视为学生。在 C++ 中，这种向上类型转换是合理且安全的，形式上可以是指针类型、引用类型和对象类型。

对于指针类型，由于子类对象中总包含父类子对象，因此将子类对象的指针视为指向父类对象的指针，在数据完整性上是足够的，在逻辑上是合理的，此时的向上类型转换是安全的。特别是在单继承下，子类对象的指针与父类对象的指针具有相同的数值，无需进行任何修改即可完成类型的转换。

对于引用类型，其内部实现本质上是通过指针完成的，因此引用类型的向上转换实际上等同于指针类型的转换，同样是安全的。

对于对象类型，将子类对象转换为父类对象也是安全的，但转换结果是一个全新的父类对象。这个新对象是通过对子类对象进行“裁剪”而生成的，只保留了父类的部分。

例如：

```

1  class Parent {
2  public:
3      Parent(int n): mNum1(n) { }
4      void f( ) { }
5  public:
6      int mNum1;
7  };
8  class Child : public Parent {
9  public:
10     Child(int n1,int n2):Parent(n1),mNum2(n2) { }
11     void g( ) { }
12 public:
13     int mNum2;
14 };
15 void k1( Parent * pObj ) { }
16 void k2( Parent & rObj ) { }
17 void k3( Parent obj ) { }
18 int main( ) {
19     Child child(1,2);    // mNum1 = 1, mNum2 = 2
20
21     Parent * ptrObj = &child; //子类指针向父类指的针转换是安全的
22     bool check1 = (ptrObj->mNum1 == 1);    //true
23     k1( &child );    //ok
24

```

```

25     Parent & refObj = child; //子类引用向父类引用的转换是安全的
26     bool check2 = (refObj.mNum1 == 1); //true
27     bool check3 = (&refObj == &child); //true
28     k2( child ); //ok
29
30     Parent obj = child; //子类对象向父类对象的转换是安全的，但经过裁剪
31     bool check4 = (obj.mNum1 == 1); //true;
32     bool check5 = (&obj == &child); //false
33     k3( child ); //ok
34 }

```

由于公有继承下的向上类型转换总是安全的，程序中任何需要父类类型的地方都可以用子类对象来替代。特别是对于指针或引用类型的向上转换，转换后的子类对象信息不会丢失，且转换效率较高，这也通常会选择使用指针或引用来作为数据成员和参数的原因之一。例如：

```

1  class Car { /* 略 */};
2  class MTCar:public Car { /* 略 */};
3  class ATCar:public Car { /* 略 */};
4  class Client {
5  public:
6      Client( Car & aCar):mpCar(&aCar) { } //实参可以是子类对象
7      Client( const Car & ) = delete;
8      Client & operator=(const Car&) = delete;
9  public:
10     void move( ) { mpCar->run( ); }
11     void compareTo(Car & otherCar) { } //实参可以是子类对象
12 private:
13     Car * mpCar = nullptr; //可以指向子类对象
14 };
15 int main( ) {
16     MTCar mtCar;
17     ATCar atCar;
18
19     Client client(mtCar); //合法
20     client.move( );
21     client.compareTo(atCar); //合法
22 }

```

在 Client 类的定义中，数据成员和成员函数原型仅涉及父类 Car，这表明在抽象层面上，Client 类仅与 Car 类交互，而不依赖于 Car 的具体子类型。即使 Car 类扩展了更多子类型，Client 类仍能保持稳定。从软件复用的角度来看，通过公有继承和向上转型，Client 类和 Car 类的稳定性得以维持，从而提高了它们的可复用性。同时，这种方式允许从 Car 类派生新的子类，即允许将 Car 类的

变化被封装在其子类中，而不影响 Client 类的功能。这充分体现了公有继承的应用价值。

16.3.2 向下类型转换

公有继承表示的是子类型化的逻辑含义，从父类向子类进行向下类型转换也是有意义的，但不同于向上类型转换，向下类型转换可能是不安全的，例如，要将一个水果转换成苹果，结果可能成功，也可能失败，转换能否成功取决于这个水果在运行时刻的实际类型。C++中可使用 `dynamic_cast` 转换运算符实现公有继承下的向下类型转换，请详见虚机制一章中的运行时类型识别。

16.4 类型转换运算符

要实现类型之间的显式转换，可以使用 C 风格的，也可以使用 C++风格的类型转换。

16.4.1 C 风格的类型转换

C 风格的类型转换是一种通用的强制类型转换方式，其格式为：

格式: *(type)expression*

使用这种类型转换时，无论在编译还是运行期间都不做类型检查，也就是转换的语义是否正确全部由程序员决定，一旦错误，容易引发未定义的行为。常用于有明确转换语义的场景。例如：

```

1  int main( ) {
2      cout << "sizeof( float ) = " << sizeof( float ) << endl;
3      cout << "sizeof( double ) = " << sizeof( double ) << endl;
4      cout << "sizeof( int ) = " << sizeof( int ) << endl;
5      cout << "sizeof( long ) = " << sizeof( long ) << endl;
6      cout << "sizeof( long long ) = " << sizeof( long long ) << endl;
7      float fpi = 3.14f;
8      //fpi 在内存中以 IEEE-754 标准格式存放(占 4 字节)
9      cout << "float pi = " << fpi << endl;
10     //fpi 在内存中实际存储的数值(十六进制表示): 4048F5C3
11     unsigned long * pFloat = (unsigned long *)&fpi;
12     cout << "float pi in Memory = " << hex << uppercase << *pFloat << endl;
13     double dpi = 3.14;
14     //dpi 在内存中以 IEEE-754 标准格式存放(占 8 字节)
15     cout << "double pi = " << dpi << endl;
16     //dpi 在内存中实际存储的数值(十六进制表示): 40091EB8-51EB851F
17     unsigned long * pLong = (unsigned long *)&dpi;
18     unsigned long long * pDouble = (unsigned long long *)&dpi;
19     cout << "double pi in Memory = " << hex << uppercase << *(pLong + 1)<<*pLong<<endl;
20     cout << "double pi in Memory = " << hex << uppercase <<*pDouble<<endl;

```

21 }

C 风格的类型转换容易产生错误，更推荐使用 C++风格的类型转换。C++语言中提供了四种类型转换运算符，分别是 `static_cast`、`const_cast`、`reinterpret_cast` 和 `dynamic_cast`。

16.4.2 `static_cast`

`static_cast` 可用于强制类型转换，其功能类似于 C 语言风格的强制类型转换。其格式为：

格式: `static_cast< type >( expression )`

使用 `static_cast` 做类型转换时，编译器可做必要的静态类型检查，从而在一定程度上保证类型转换的合法性，减少由类型转换引起的程序错误。因此，虽然 `static_cast` 在语法上麻烦些，但规范中仍推荐使用 `static_cast` 做类型的强制转换，而尽量不使用 C 语言风格的强制类型转换。例如：

```

1  #include <iostream>
2  class Base { };
3  class Derived : private Base { };
4  class Fruit {
5  public: virtual ~Fruit( ) { }
6  };
7  class Apple :public Fruit { };
8  class Orange :public Fruit { };
9  int main( ) {
10     double val = 3.14;    //sizeof(double) == 8, sizeof(unsigned long) == 4
11     unsigned long * pLong = (unsigned long *)(&val);    //合法, C 风格
12     //unsigned long * pLong2 = static_cast<unsigned long *>( &val ); //非法, C++风格
13     long n1 = (long)val;    //合法, C 风格
14     long n2 = static_cast<long>(val); //合法, C++风格, 推荐使用
15     Derived d;
16     Base * pBase1 = (Base *)(&d);    //合法, C 风格
17     //Base * pBase2 = static_cast< Base * >( &d );    //非法, C++风格
18     Apple apple;
19     Fruit & fruit = apple;
20     Orange * pOrange1 = (Orange *)(&fruit);    //合法, C 风格, 但高风险
21     Orange * pOrange2 = static_cast<Orange *>(&fruit); //合法, C++风格, 但高风险
22     Orange * pOrange3 = (Orange *)(&apple);    //合法, C 风格, 但高风险
23     //Orange * pOrange4 = static_cast<Orange *>( &apple );    //非法, C++风格
24     //使用 pOrange1、pOrange2 和 pOrange3 有潜在的高风险, 可产生运行时错误
25 }
```

从例子中可以看出，使用 C 风格的类型转换，在时间和地点上没有过多的限制，其使用风险完全由程序员负责。使用 `static_cast` 只做编译期的类型检查，不进行运行时的类型检查，使用的风险

小于使用 C 风格转换。

16.4.3 const_cast

const_cast 用于添加或移除 const 或 volatile 修饰符。其格式为：

格式: `const_cast< type >( expression )`

当使用 const 修饰符时,我们通常称之为“常量”,但更准确地说,它表示“稳定”。相对地,volatile 修饰符与 const 相反,表示“易变”,通常用于多线程或存在中断的程序中。对于带有 volatile 修饰符的变量,编译器将禁止进行缓存等优化处理,确保每次访问 volatile 变量时都是直接从内存中读取或写入。例如:

```

1  class A {
2  public:
3      A(int n):val(n) { }
4      void func1( ) {
5          //由于 val 带有 volatile 修饰, 编译器不会将下面三条语句优化成一条 val = 3;
6          val = 1;
7          val = 2;
8          val = 3;
9      }
10     int func2(volatile int & k) {
11         //实现 1,每次都到内存中读取变量 k 的值, 返回的不一定是 k 的平方
12         return k*k;
13         //实现 2,真正实现计算 k 的平方
14         int temp = k;
15         return temp*temp;
16         //实现 3,去掉了 volatile 修饰, 真正实现计算 k 的平方
17         int & temp = const_cast<int &>(k);
18         return temp*temp;
19     }
20 private:
21     volatile int val;
22     int other;
23 };

```

一般来说,添加 const 或 volatile 修饰符总是允许的,无需特殊运算符。然而,移除 const 或 volatile 则受到限制,这时 const_cast 运算符就显示出作用了。例如:

```

1  int main( ) {
2      int a = 100;
3      //允许添加 const 或 volatile

```

```

4     const int ca = a;
5     volatile int va = a;
6     //移除 const 或 volatile
7     //int & noConstInt = ca;           //非法, 禁止直接移除 const 修饰
8     //int & noVolatileInt = va;        //非法, 禁止直接移除 volatile 修饰
9     int & noConstInt = const_cast<int &>(ca);    //合法地移除 const 修饰
10    int & noVolatileInt = const_cast<int &>(va);   //合法地移除 volatile 修饰
11    Base base;
12    //允许添加 const 或 volatile
13    const Base & cBase = base;
14    volatile Base & vBase = base;
15    //移除 const 或 volatile
16    //Base & noConstBase = cBase;       //非法, 禁止直接移除 const 修饰
17    //Base & noVolatileBase = vBase;    //非法, 禁止直接移除 volatile 修饰
18    Base & noConstBase = const_cast<Base &>(cBase); //合法地移除 const 修饰
19    Base & noVolatileBase = const_cast<Base &>(vBase); //合法地移除 volatile 修饰
20 }

```

下面是一个更贴近实际使用 `const_cast` 运算符的例子:

```

1  #include <iostream>
2  struct A {
3      A( int n ) :mNum( n ) { }
4      void func( ) const { //带 const 修饰的成员函数, 也可修改当前对象
5          A * pA = const_cast< A * >( this );
6          ++pA->mNum;
7      }
8      void setNum( int n ) { mNum = n; }
9      int mNum = 0;
10 };
11 int main( ) {
12     const A a( 1 );
13     std::cout << "a.mNum = " << a.mNum << std::endl;    //输出 a.mNum = 1
14     a.func( );    //a 是常对象, func( )是常成员函数, 但可修改!!
15     std::cout << "a.mNum = " << a.mNum << std::endl;    //输出 a.mNum = 2
16     //a.setNum( 100 );    //错误, 常对象不允许修改
17     A * pA = const_cast< A * >( &a );
18     pA->setNum( 100 );
19     std::cout << "a.mNum = " << a.mNum << std::endl;    //输出 a.mNum = 100
20 }

```

在例子中, `A::func()` 是一个带有 `const` 修饰的成员函数。通常情况下, 这类函数不允许修改对象的实例变量。如果程序员确实需要修改当前对象, 而又不想去掉函数原型中的 `const` 修饰符,

可以像示例中那样使用 `const_cast` 来实现。同样地，在 `main` 函数中，对象 `a` 是一个常量对象，因此不能直接调用非 `const` 成员函数 `setNum`。如果确实需要调用该函数，可以通过 `const_cast` 移除对象的 `const` 修饰符后再进行访问。

16.4.4 reinterpret_cast

`reinterpret_cast` 通过重新解释底层的位模式来实现类型之间的转换。其格式为：

格式: `reinterpret_cast< type >( expression )`

在使用 `reinterpret_cast` 转换为新类型的过程中，数据在内存中的二进制表示不会发生改变，只是告诉编译器以新的类型来对待目标数据。例如：

```

1  #include <iostream>
2  struct A    {    int mA = 1;  };
3  struct B    {    int mB = 2;  };
4  struct C : A {    int mC = 3;  };
5  struct D : A, B { int mD = 4;  };
6  int f( int n ) {
7      std::cout << n << std::endl;
8      return n + 100;
9  }
10 unsigned short hash( void * p ) {
11     unsigned int n = reinterpret_cast< unsigned int >( p );
12     return ( unsigned short ) ( n ^ ( n >> 16 ) );
13 }
14 int main( ) {
15     //取得单精度浮点数在内存中存储的二进制表示
16     float val = 3.14f;
17     unsigned long * pUlong1 = ( unsigned long * ) ( &val );    //合法，C 风格类型转换
18     //unsigned long ulVal = static_cast<unsigned long *>(&val); //非法，C++风格强制转换
19     unsigned long * pUlong2 = reinterpret_cast< unsigned long * >( &val ); //合法 C++风格
20     std::cout << std::hex << *pUlong1 << " " << *pUlong2 << std::endl;
21     //子类向父类转换
22     D d;
23     A * pA = static_cast< A * >( &d );
24     std::cout << "pA->mA =" << pA->mA << std::endl; //输出 1
25     B * pB = static_cast< B * >( &d );                //static_cast 做内部额外处理
26     std::cout << "pB->mB =" << pB->mB << std::endl; //输出 2
27     B * pB1 = ( B * ) ( &d );                        //C 风格强制转换，也会做内部额外处理
28     std::cout << "pB1->mB =" << pB1->mB << std::endl; //输出 2
29     C * pC = reinterpret_cast< C * >( &d );           //合法，但不保证安全!!!
30     //static_cast< C * >( &d );                       //非法

```

```

31     std::cout << "pC->mC =" << pC->mC << std::endl; //输出 2
32     //将函数 int f(int); 重新解释成 void f(int);
33     using voidFunc = void ( * )( int );
34     voidFunc g = reinterpret_cast< voidFunc >( f ); //不做安全保证
35     g( 100 );           //不做安全保证
36     //输出指针的 hash 值
37     int arr[ 10 ];
38     for ( int i = 0; i < 10; i++ ) {
39         std::cout << hash( arr + i ) << std::endl;
40     }
41 }

```

从示例可以看出, `reinterpret_cast` 可用于不同类型之间的转换, 但它不会改变内存数据的布局和二进制值。因此, 使用 `reinterpret_cast` 存在风险, 编译器既不能保证其安全性, 也无法保证平台间的可移植性。请务必谨慎使用!

16.4.5 `dynamic_cast`

`dynamic_cast` 用于在公有继承中实现运行时的向下类型转换, 即将父类的指针或引用安全地转换为子类的指针或引用。

在公有继承关系中, 将子类对象的指针或引用转换为父类的指针或引用总是安全的。然而, 将父类的指针或引用转换为子类的指针或引用则不一定安全, 这取决于对象的动态类型。为了确保这种转换的安全性, 需要使用 `dynamic_cast` 运算符。有关详细讨论, 请参阅 18.8.2 `dynamic_cast` 运算符。